

CSC490 Assignment A5

Testing Your Project

Ananya Jain (1007706573), Aviral Bhardwaj (1008239407)
Ansh Agrawal (1007413103), Mann Thakkar (1007647744)

March 28, 2026

1 Part One: Profiling Execution Time

Summary of Timings (CPU Compute Bound)

Function	Slow (injected)	Optimized (current)	Speedup
ShardDataset.__getitem__	0.130s / 500 calls	0.005s / 500 calls	~26x
LoRAMoELayer.forward	8.374s / 100 calls	7.357s / 100 calls	~1.1x
StressCorrectedRouter.forward	3.387s / 300 calls	0.340s / 300 calls	~10x
MetabolicRouter.forward	3.114s / 300 calls	0.328s / 300 calls	~9.5x
GlobalSpecializationTracker.update	4.256s / 300 calls	0.130s / 300 calls	~33x

1.1 ShardDataset.__getitem__

1.1.1 Analysis

This function is a critical data-loading path executed for every training sample. The main inefficiencies came from linearly scanning shard boundaries, reopening memory-mapped files on every call, and performing redundant per-token Python operations.

1.1.2 Before

```
# O(N) linear scan
shard_idx = 0
for i, cum in enumerate(self.cumulative):
    if cum <= pos:
        shard_idx = i

# Re-open memmap every call
fresh_mmap = np.memmap(self.shards[shard_idx], dtype=dtype, mode="r", ...)
chunk = fresh_mmap[local_offset : local_offset + n].astype(np.int64)

# Redundant per-token Python loop
for i in range(len(tokens)):
    _ = int(tokens[i])
```

1.1.3 After

```
# O(log N) binary search
shard_idx = bisect.bisect_right(self.cumulative, pos) - 1

# Cached mmap
chunk = self.mmmaps[shard_idx][local_offset : local_offset + n].astype(np.int64)
```

1.1.4 Improvement

Lookup became faster and file I/O was removed from the hot path.

1.2 LoRAMoELayer.forward

1.2.1 Analysis

This function handles expert dispatch in a Mixture-of-Experts layer. The inefficiency came from unnecessary tensor copies and repeated memory allocations inside the expert loop.

1.2.2 Before

```
expert_in = expert_in.clone().contiguous()

tmp = torch.zeros_like(combined)
tmp[token_ids] = expert_out * scale
combined = combined + tmp
```

1.2.3 After

```
expert_in = x_flat[token_ids]
expert_out = expert(expert_in)
combined[token_ids] += expert_out * scale
```

1.2.4 Improvement

This removed extra copies and reduced memory overhead.

1.3 StressCorrectedRouter.forward

1.3.1 Analysis

This routing function computes cosine similarity between tokens and expert weights. The original implementation used nested Python loops with repeated norm computations, which created heavy Python overhead.

1.3.2 Before

```
for i in range(B):
    for j in range(S):
        token = x[i, j]
        norm_val = token.norm(p=2).clamp(min=1e-12)
        ...
```

1.3.3 After

```
x_norm = F.normalize(x, dim=-1)
W_norm = F.normalize(self.W, dim=-1)
cos_sim = x_norm @ W_norm.T
```

1.3.4 Improvement

This replaced many small operations with batched kernels.

1.4 MetabolicRouter.forward

1.4.1 Analysis

This function had the same normalization issue as the previous router, along with extra tensor stacking overhead.

1.4.2 Before

```
for i in range(B * S):
    flat = ...
    n = flat.norm(...)
    ...
x_norm = torch.stack(...)
```

1.4.3 After

```
x = F.normalize(x, dim=-1)
w = F.normalize(self.gate.weight, dim=-1)
return F.linear(x, w)
```

1.4.4 Improvement

This reduced the work to a small number of vectorized operations.

1.5 GlobalSpecializationTracker.update

1.5.1 Analysis

This function updates token-expert usage statistics. The inefficiency came from iterating over every token-expert pair in Python and calling `.item()` repeatedly.

1.5.2 Before

```
for i in range(n_tokens):
    tok = expanded_tokens[i].item()
    exp = flattened_experts[i].item()
    self.usage_counts[tok, exp] += 1
```

1.5.3 After

```
linear_indices = expanded_tokens * self.num_experts + flattened_experts
counts = torch.bincount(linear_indices, minlength=...)
self.usage_counts += counts.view(...)
```

1.5.4 Improvement

This vectorized the update and removed Python-loop overhead.

2 Part Two: Showing Code Coverage

To measure and report code coverage, we used `pytest-cov` integrated with a GitHub Actions workflow. All tests are automatically executed on every push and pull request, ensuring continuous validation of the codebase.

2.1 Coverage Visualization

To improve visibility, coverage results are displayed both in pull requests and in the project README.

- **Pull Request Coverage Report:** Each pull request automatically includes a comment summarizing the test coverage results. This allows contributors to quickly see how their changes impact coverage before merging.
- **README Coverage Badge:** The README is dynamically updated with a coverage badge that reflects the latest test coverage percentage from the main branch.

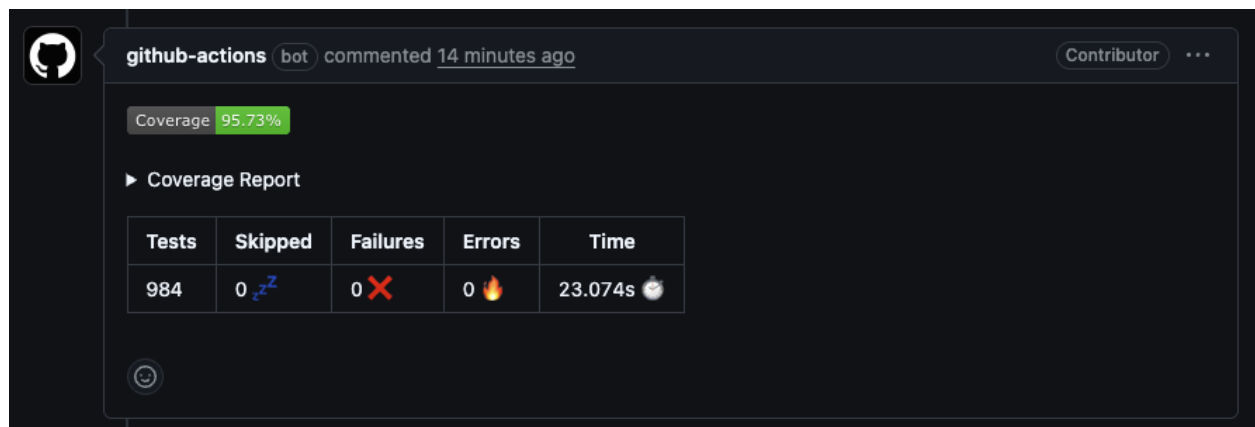


Figure 1: Coverage summary automatically posted on a pull request.

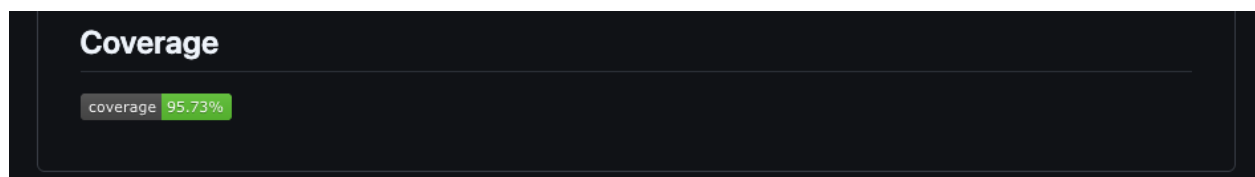


Figure 2: Coverage badge displayed in the project README.

3 Part Three: Test Coverage Breadth

To evaluate the breadth of testing across the entire codebase, we used the `pytest-cov` tool to measure code coverage. All tests were executed using the following command:

```
pytest --cov=. --cov-report=term-missing
```

This command measures coverage across the entire repository, including source code, scripts, and entrypoint files. The test suite consists of over 1000 unit and integration tests, ensuring that a wide range of functionality is exercised.

3.1 Coverage Results

The final coverage summary is:

Total Coverage: 95.73%

This indicates that the vast majority of the codebase is executed during testing, including core modules, infrastructure components, and script-level logic. Only a small fraction of lines remain uncovered, typically corresponding to rare edge conditions or defensive branches.

For example, certain entrypoint files such as `run_modal_training.py` and `train.py` were not fully unit tested, as they depend on external infrastructure (e.g., Modal cloud execution, GPU hardware, distributed training, and filesystem shards) that cannot be reliably reproduced in a local test environment.

4 Part Three: Test Coverage Depth

4.1 Chosen Module

We selected `src/metrics/router_metrics.py` as the most complex module in the codebase. It contains two major classes, `GlobalSpecializationTracker` and `RouterMetricsTracker`, and implements several non-trivial routing metrics including entropy, Gini coefficient, effective experts, collapse score, specialization score, and confidence-based statistics. The module also includes multiple conditional branches for dense versus sparse routing, distributed versus non-distributed synchronization, and router-specific metric outputs. In addition, it contains several numerical edge cases such as zero usage, degenerate distributions, and invalid routing values. For these reasons, it was the most suitable module for a deep unit-test suite.

4.2 Why This Module Was Chosen

This module was chosen because it is central to model observability and contains the greatest concentration of branching logic in the metrics layer. Small implementation errors in this file could silently produce incorrect statistics without causing immediate failures. The test suite therefore focuses on correctness, stability, and robustness across both normal and adversarial inputs.

4.3 Test Suite Summary

The file `test_router_metrics.py` contains 62 unit tests organized into 20 test classes. The tests were designed to cover both positive cases and negative cases, with emphasis on edge conditions, failure modes, and subtle numerical behavior. Each test group is documented with comments explaining the scenario it targets and why that scenario matters.

4.4 Representative Test Coverage

Test case	Type	Positive / Negative	Purpose
Dense routing usage computation	Unit test	Positive	Verifies that the dense branch of <code>_compute_usage</code> correctly aggregates expert weights when no routing indices are provided.
Sparse routing with negative indices	Unit test	Negative	Checks that invalid <code>-1</code> expert indices are safely handled and do not corrupt usage counts.
Gini coefficient correctness	Unit test	Positive	Confirms mathematically correct Gini values for known distributions such as uniform routing and monopoly routing.
Entropy stability for degenerate usage	Unit test	Negative	Ensures all-zero or near-zero usage values do not produce NaN or infinite values.
Effective experts identity	Unit test	Positive	Verifies that effective experts equals <code>exp(entropy)</code> and that the entropy value is reused consistently.
Top-1 dominance boundary case	Unit test	Positive	Confirms that when <code>top_k=1</code> , the top-1 dominance metric behaves as expected.
Shared usage consistency in <code>compute_all_metrics</code>	Unit test	Positive	Ensures entropy, Gini, and effective experts are computed from the same usage tensor.
Router-specific metric keys	Unit test	Positive	Verifies that only the expected router-specific fields are included for each router type.
Padding token filtering	Unit test	Negative	Ensures tokens outside the valid vocabulary range are ignored rather than counted as valid data.
Padding expert filtering	Unit test	Negative	Confirms that invalid expert IDs are filtered before histogram accumulation.
Accumulation across updates	Unit test	Positive	Checks that repeated calls to <code>update()</code> accumulate state instead of resetting it.
Specialization score behavior	Unit test	Positive and Negative	Tests both highly specialized and near-random routing cases, including a degenerate marginal-entropy branch.
Non-destructive synchronization	Unit test	Positive	Verifies that <code>sync_and_compute()</code> returns metrics without modifying local tracker state.
Fatigue statistics handling	Unit test	Positive and Negative	Confirms that fatigue metrics are present only when the router supports them, and absent otherwise.

Test case	Type	Positive / Negative	Purpose
Normalized usage distribution	Unit test	Positive	Ensures the usage distribution sums to one and remains finite even when usage is zero.
Single-expert boundary case	Unit test	Positive	Validates that <code>num_experts=1</code> produces the expected metric values without numerical issues.
Empty tracker state	Unit test	Negative	Confirms that metrics can be queried before any updates without causing a crash.
Gini cache behavior	Unit test	Positive	Verifies that the cached Gini result is populated and reused consistently.
Global token counting	Unit test	Positive	Ensures that token counts are tracked correctly and are not inflated by routing multiplicity.
Large batch stability	Unit test	Negative	Stress-tests the module with larger inputs to confirm stability and absence of shape or numerical errors.

4.5 Why These Tests Were Selected

These tests were chosen to cover the main classes of errors that are most likely to occur in this module: incorrect branching, numerical instability, invalid input handling, and incorrect state accumulation. The suite includes both normal use cases and edge cases so that the implementation is tested from multiple angles rather than only on the happy path.

4.6 Repository Links

- Test file: [tests/test_router_metrics.py](#)
- Module under test: [src/metrics/router_metrics.py](#)

5 Part Four: Two Memorable Bugs

Bug 1: Final Checkpoint Not Saved Without Validation Loader

Video: <https://drive.google.com/file/d/1bvqiNrTDv90xD5VMqOLKEqEztIKJT5ZI/view?usp=sharing>

What the bug was and how I discovered it

The final checkpoint save at the end of training was nested inside a `if val_loader is not None` conditional block, meaning that when training was run without a validation dataset, the final model state was never written to disk. I discovered this after a full training run on Modal completed successfully but the final checkpoint was absent from the output directory, with only periodic interval checkpoints present.

Debugging process

I first verified that the output directory and checkpoint subdirectory were being created correctly, which they were. I then confirmed that intermediate checkpoints from the periodic `save_interval` were present, ruling out a permissions or path issue. Tracing through the end-of-training code, I

identified that the `save_checkpoint` call was placed inside the same conditional branch responsible for computing the final validation loss, rather than being separated from it.

Fix

I restructured the end-of-training block so that the checkpoint save is unconditional. The validation loss computation and best-model flag update remain gated on `val_loader is not None`, but the `save_checkpoint` call is moved outside that block so the final model state is always persisted regardless of the training configuration.

What I learned

Critical side effects such as model persistence should not be implicitly coupled to optional features. The presence or absence of a validation set is a data configuration concern and should have no bearing on whether the final training artifact is saved. Conflating these two responsibilities made the failure mode silent and difficult to detect.

Test cases

I added a test that runs a short training loop with no validation shards, allows it to complete, and asserts that a final checkpoint file exists at the expected path. This directly encodes the invariant that training completion must always produce a saved checkpoint, independent of the validation configuration.

Bug 2: `dataset_exists()` Causes Re-ingestion

Video: https://drive.google.com/file/d/17Eg-AmedvvHNRaevGzWFElkoh7MOuZl_/view?usp=drive_link

What the bug was and how I discovered it

The `dataset_exists()` guard always returned `False` for an already-ingested dataset, causing repeated downloads and S3 uploads. I noticed this from unusually high S3 PUT requests and CloudWatch logs showing ingestion running on every job.

Debugging process

I first checked if the S3 prefix was incorrect, but paths matched. Then I verified `list_objects()` and confirmed files were being returned. This led me to inspect the filtering logic inside `dataset_exists()`.

Fix

The extension check was incomplete and did not include all supported formats. I updated it to include the missing extension so detection matches ingestion.

What I learned

Small inconsistencies between components can cause large performance and cost issues. Shared assumptions (like supported formats) should be centralized.

Test cases

I added a unit test mocking S3 to ensure `dataset_exists()` returns `True` when valid dataset files exist.